

# 드라마 편성표 앱 아키텍처 설계 문서

DramaApp · 1인 개발자를 위한 실전 아키텍처 가이드

## 핵심 의사결정 원칙

1. MVP는 버릴 수 있게, 코어 도메인은 깨지지 않게.
2. 혼자서 운영 가능한 만큼만 기능을 연다.

프로젝트: DramaApp (iOS)

작성일: 2026-05-24

대상: 1인 개발 시작 → 1년 내 10만 MAU 시나리오

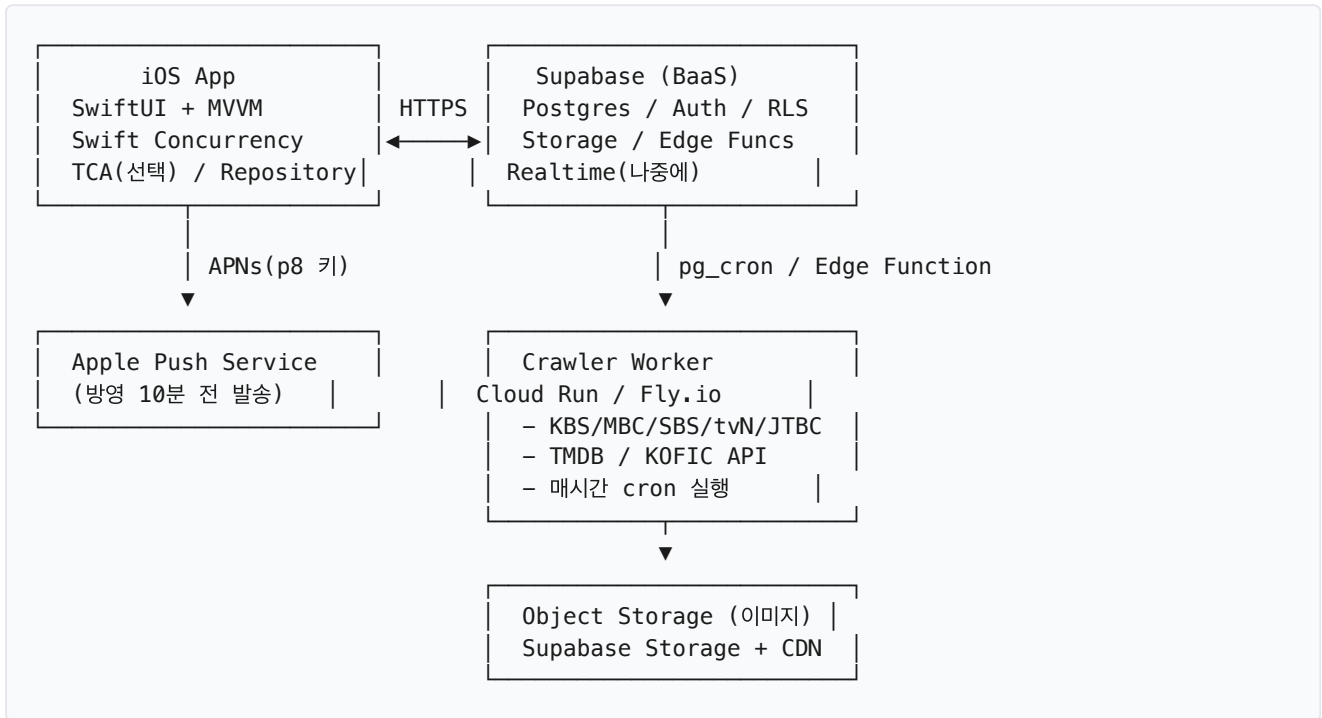
# 목차

---

1. 앱 전체 구조
2. MVP 우선순위
3. 화면 구성
4. DB 테이블 설계
5. API 설계
6. 추천 기술 스택
7. 서버 아키텍처
8. 푸시 알림 구조
9. 커뮤니티 설계
10. 드라마 데이터 수집 전략
11. 예상 트래픽 고려사항
12. 비용 절감 전략
13. 추후 수익화 아이디어
14. 앱스토어 출시 시 주의사항
15. 실제 서비스 운영 시 가장 어려운 점
16. 혼자 개발할 경우 현실적인 개발 순서
17. 3개월 개발 로드맵
18. 확장 가능한 구조 설계 방법
19. 최종 요약 — 오늘 당장 할 일

# 1. 앱 전체 구조

## 1-1. 시스템 레벨 구성도



## 1-2. iOS 앱 아키텍처

선택: SwiftUI + MVVM + Repository 패턴 (TCA는 도입하지 않음).

| 기준              | MVVM  | TCA   |
|-----------------|-------|-------|
| 학습 곡선           | 낮음    | 높음    |
| 테스트 용이성         | 중간    | 매우 높음 |
| 보일러플레이트         | 적음    | 많음    |
| 1인 개발 적합성       | ★★★★★ | ★★    |
| 화면 30개 이상 시 일관성 | 중간    | 매우 좋음 |

**이유:** 1인 개발에서 TCA는 가속이 아니라 감속입니다. 화면이 30개를 넘기 시작하고 상태가 얹히기 시작하면 그때 **부분 도입**(Feature 단위)이 더 현실적입니다. 처음부터 TCA를 깔면 단순한 즐겨찾기 토글 하나에도 Action/Reducer를 그려야 합니다.

Clean Architecture는 "축소판"으로만. 3-layer로 충분합니다:

- **Presentation:** SwiftUI View + ViewModel( `@Observable` )

- Domain: **Drama** , **Episode** , **Schedule** 모델 + **...UseCase** (옵션, 복잡한 흐름만)
- Data: **Repository** 프로토콜 + **SupabaseDramaRepository** 구현체

레포지토리를 프로토콜로 끊어둬야 나중에 Supabase → 자체 백엔드(NestJS)로 갈아낄 때 ViewModel을 안 건드릴 수 있습니다.

### 1-3. 폴더 구조 예시

```

DramaApp/
├── App/
│   ├── DramaAppApp.swift           // @main, SwiftUI App
│   ├── AppDependencies.swift       // DI 컨테이너
│   └── RootView.swift              // 탭 컨테이너
├── Features/
│   ├── Schedule/                   // 편성표
│   │   ├── ScheduleView.swift
│   │   ├── ScheduleViewModel.swift
│   │   └── Components/
│   ├── Favorites/
│   ├── Search/
│   ├── Community/
│   │   ├── BoardList/
│   │   ├── PostDetail/
│   │   └── Composer/
│   └── Profile/
├── Domain/
│   ├── Models/                    // Drama, Episode, Channel, Post...
│   └── UseCases/                  // ToggleFavoriteUseCase 등 (필요할 때만)
├── Data/
│   ├── Repositories/
│   │   ├── DramaRepository.swift    // 프로토콜
│   │   └── SupabaseDramaRepository.swift // 구현체
│   ├── Network/
│   │   ├── SupabaseClient+Shared.swift
│   │   └── APIError.swift
│   └── Local/
│       ├── FavoritesStore.swift      // SwiftData
│       └── ScheduleCache.swift       // 디스크 캐시
├── Push/
│   ├── PushService.swift
│   └── NotificationCategory.swift
├── DesignSystem/
│   ├── Colors.swift
│   ├── Typography.swift
│   └── Components/                 // DramaCard, EpisodeRow 등
└── Resources/
    ├── Localizable.xcstrings       // String Catalog
    └── Assets.xcassets

```

Feature-first 폴더링을 강력 추천합니다. Layer-first( **Views/** , **ViewModels/** 따로)는 화면이 늘어나면 한 기능 수정에 폴더 5개를 왔다갔다 해야 합니다.

추후 사용자가 늘어 빌드가 느려지면(>2분), 각 **Feature/** 를 SPM 로컬 패키지로 분리합니다. 이때를 위해 **Feature** 간 직접 **import**를 금지하고 모두 **Domain** 을 통해 통신하도록 처음부터 규칙을 잡으세요.

## 2. MVP 우선순위

"매일 들어오는 플랫폼"이 목표라면 **재방문을 만드는 기능부터** 갑니다. 커뮤니티는 사용자가 모이기 전에는 죽은 공간이고, 죽은 게시판은 신규 사용자를 쫓아냅니다.

| 우선순위 | 기능                 | 이유                    | 출시 시점 |
|------|--------------------|-----------------------|-------|
| P0   | 편성표 조회(채널/날짜)      | 앱 핵심 가치. 이게 없으면 앱이 아님 | v1.0  |
| P0   | 즐거찾기 + 로컬 저장       | 재방문 동기. 서버 없이도 동작 가능  | v1.0  |
| P0   | 방영 10분 전 푸시        | "왜 매일 켜는가"의 핵심        | v1.0  |
| P0   | Apple 로그인 (게스트 포함) | 즐거찾기 동기화 + 4.8 가이드 대응 | v1.0  |
| P1   | 검색(드라마명)           | 편성표만으로 신규 드라마 찾기 불편   | v1.1  |
| P1   | 장르 필터              |                       | v1.1  |
| P1   | 상세 화면(시놉시스/회차)     | TMDB 데이터로 빠르게         | v1.1  |
| P2   | 커뮤니티(게시판/댓글)       | 사용자 1만 모인 후           | v2.0  |
| P2   | 신고/차단/블라인드         | 커뮤니티와 동시              | v2.0  |
| P3   | 인기글, 활동 기록         |                       | v2.x  |
| P3   | 배우 기반 검색, 출연작 알림   | TMDB person 데이터 필요    | v3.0  |
| P4   | 실시간 채팅             | 운영 부담 큼. 정말 필요할 때     | v4.0+ |

**핵심:** P0 4개만 정확히 구현해도 "편성표 + 알림" 시장에서 충분히 경쟁력 있습니다. 한국에서 드라마 알림 시장은 비어 있습니다. 커뮤니티는 후행 기능입니다.

### 3. 화면 구성

탭바 5개 구조: [편성표] [즐거찾기] [검색] [커뮤니티] [마이]

#### 3-1. 편성표 탭 (Schedule)

← 5월 24일(토) →

[채널 ▼]

← 날짜 스와이프, 채널 필터

|         |      |                 |                 |
|---------|------|-----------------|-----------------|
| 19:30   | KBS2 | [LIVE] 드라마A 12회 | ← 현재 방영중은 빨간 배지 |
| ♡ 알림 ON |      |                 |                 |
| 20:40   | MBC  | 드라마B 8회         |                 |
| ♡ 알림    |      |                 |                 |
| 21:50   | SBS  | 드라마C (첫방)       |                 |
| ♡ 알림    |      |                 |                 |

상단: 가로 스크롤 가능한 날짜 캐러셀(±14일). 우측 상단: 채널 필터(다중 선택). 행 탭 → 드라마 상세.

#### 3-2. 즐겨찾기 탭 (Favorites)

내 드라마 (12)

[편집]

|                   |
|-------------------|
| [● NEW] 드라마A      |
| 오늘 19:30 12회 방영   |
| 드라마B              |
| 내일 20:40 9회 (D-1) |
| 드라마C (종영)         |
| 전체 16회 ▶          |

상태 그룹: 오늘 방영 / 내일 / 이번주 / 종영. 종영작은 별도 섹션으로 내려서 활성 드라마에 집중시킵니다.

#### 3-3. 검색 탭

상단: 검색바. 하단: 인기 검색어 + 장르 칩. 결과는 드라마/배우 탭 분리(배우는 v3).

### 3-4. 드라마 상세

[포스터 이미지]

드라마A · 2026 · KBS2

♡ 즐겨찾기  알림 받기

[회차] [정보] [출연] [게시판]

← 세그먼트

12회 5/24 19:30

11회 5/23 19:30 ▶

...

### 3-5. 커뮤니티 (드라마 게시판)

게시글 카드: 제목 / 작성자(닉네임) / 좋아요·댓글 수 / **스포일러 토글**. 스포 게시글은 회색 마스크 + "탭해서 보기".

### 3-6. 마이

프로필(닉네임, 가입일), 내 활동(쓴 글/댓글), 알림 설정, 차단 목록, 로그아웃, **회원 탈퇴(4.8 필수)**.

## 4. DB 테이블 설계

PostgreSQL 기준. 모든 PK는 `uuid` (Supabase 관행), 모든 테이블에 `created_at`, `updated_at`, 필요시 `deleted_at` (소프트 삭제).

### 4-1. ERD (텍스트)

```
users —< favorites >— dramas —< episodes
|
|—< drama_casts >— persons
|—< drama_genres >— genres
|
|—< devices (푸시 토큰)
|—< posts >— comments —< comment_likes
|   |
|   |—< post_likes
|—< reports
|—< blocks
```

## 4-2. 핵심 DDL 예시

```
-- 1. 사용자
create table users (
  id          uuid primary key default gen_random_uuid(),
  apple_sub   text unique not null,
  nickname    text unique not null,
  avatar_url  text,
  is_blocked  boolean default false,
  created_at  timestampz default now(),
  updated_at  timestampz default now()
);

-- 2. 채널
create table channels (
  id          uuid primary key default gen_random_uuid(),
  code        text unique not null,    -- 'KBS2','MBC','SBS','tvN','JTBC','NETFLIX'
  name        text not null,
  type        text not null,          -- 'BROADCAST' | 'OTT'
  logo_url    text
);

-- 3. 드라마
create table dramas (
  id          uuid primary key default gen_random_uuid(),
  title       text not null,
  title_en    text,
  synopsis    text,
  poster_url  text,
  channel_id  uuid references channels(id),
  status      text not null,          -- 'UPCOMING' | 'ON_AIR' | 'ENDED'
  total_episodes int,
  start_date  date,
  end_date    date,
  tmdb_id     int,
  external_id text,
  created_at  timestampz default now(),
  updated_at  timestampz default now()
);
create index idx_dramas_status on dramas(status);
create index idx_dramas_channel on dramas(channel_id);
create index idx_dramas_title_trgm on dramas using gin (title gin_trgm_ops);

-- 4. 회차
create table episodes (
  id          uuid primary key default gen_random_uuid(),
  drama_id    uuid not null references dramas(id) on delete cascade,
  number      int not null,
  air_time    timestampz not null,
  duration_min int default 70,
  is_special  boolean default false,
  created_at  timestampz default now(),
  unique (drama_id, number)
);
create index idx_episodes_air_time on episodes(air_time);
```

```

create index idx_episodes_air_date on episodes((air_time::date));

-- 5. 즐겨찾기
create table favorites (
  user_id    uuid not null references users(id) on delete cascade,
  drama_id   uuid not null references dramas(id) on delete cascade,
  notify_air boolean default true,
  notify_new_episode boolean default true,
  created_at timestamptz default now(),
  primary key (user_id, drama_id)
);

-- 6. 디바이스
create table devices (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid references users(id) on delete cascade,
  apns_token  text unique not null,
  app_version text,
  os_version  text,
  locale      text default 'ko-KR',
  last_seen_at timestamptz default now(),
  created_at  timestamptz default now()
);

-- 7. 게시물
create table posts (
  id          uuid primary key default gen_random_uuid(),
  drama_id    uuid not null references dramas(id) on delete cascade,
  user_id     uuid not null references users(id),
  title       text not null,
  body        text not null,
  is_spoiler  boolean default false,
  episode_no  int,
  like_count  int default 0,      -- 비정규화 (성능)
  comment_count int default 0,   -- 비정규화
  hot_score   real default 0,    -- 인기글 점수
  is_hidden   boolean default false,
  created_at  timestamptz default now(),
  updated_at  timestamptz default now()
);
create index idx_posts_drama_created on posts(drama_id, created_at desc);
create index idx_posts_hot on posts(hot_score desc) where is_hidden = false;

-- 8. 댓글
create table comments (
  id          uuid primary key default gen_random_uuid(),
  post_id     uuid not null references posts(id) on delete cascade,
  user_id     uuid not null references users(id),
  parent_id   uuid references comments(id),
  body        text not null,
  is_spoiler  boolean default false,
  like_count  int default 0,
  is_hidden   boolean default false,
  created_at  timestamptz default now()
);

-- 9. 좋아요

```

```

create table post_likes (
  post_id  uuid not null references posts(id) on delete cascade,
  user_id  uuid not null references users(id) on delete cascade,
  created_at timestampz default now(),
  primary key (post_id, user_id)
);

-- 10. 신고
create table reports (
  id          uuid primary key default gen_random_uuid(),
  reporter_id uuid not null references users(id),
  target_type text not null,      -- 'POST' | 'COMMENT' | 'USER'
  target_id   uuid not null,
  reason      text not null,
  detail      text,
  status      text default 'PENDING',
  created_at  timestampz default now()
);

-- 11. 차단
create table blocks (
  blocker_id uuid not null references users(id) on delete cascade,
  blocked_id uuid not null references users(id) on delete cascade,
  created_at timestampz default now(),
  primary key (blocker_id, blocked_id)
);

-- 12. 알림 큐
create table push_jobs (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid references users(id) on delete cascade,
  episode_id  uuid references episodes(id) on delete cascade,
  scheduled_at timestampz not null,
  status      text default 'PENDING',
  payload     jsonb not null,
  created_at  timestampz default now(),
  sent_at     timestampz
);
create index idx_push_jobs_due on push_jobs(scheduled_at) where status = 'PENDING';

```

#### 중요 포인트

- `like_count`, `comment_count` 는 비정규화 필드입니다. 좋아요할 때마다 `count(*)` 를 돌리면 DB가 죽습니다. 트리거 또는 트랜잭션에서 증감.
- `episodes.air_time` 은 반드시 **timestampz (UTC 저장)**. 클라이언트에서 KST 변환.
- `dramas.title` 에 `pg_trgm` 인덱스로 한글 부분 검색 지원.
- 모든 PK는 `uuid` — 분산 환경에서 ID 충돌 방지, 추측 불가(보안).

### 4-3. RLS (Row-Level Security) 예시

Supabase는 RLS가 필수입니다. 안 켜면 익명 사용자가 다 읽고 씁니다.

```
alter table posts enable row level security;

create policy "posts_select_all" on posts
  for select using (is_hidden = false);

create policy "posts_insert_own" on posts
  for insert with check (auth.uid() = user_id);

create policy "posts_update_own" on posts
  for update using (auth.uid() = user_id);

create policy "posts_hide_blocked" on posts
  for select using (
    not exists (
      select 1 from blocks
      where blocker_id = auth.uid() and blocked_id = posts.user_id
    )
  );
```

## 5. API 설계

Supabase를 쓰면 대부분의 CRUD는 **PostgREST**가 자동 생성해주므로 직접 REST 설계는 **복잡한 도메인 로직**에만 적용합니다.

### 5-1. Supabase 자동 API (직접 사용)

```
# 오늘 편성표
GET /rest/v1/episodes?
  air_time=gte.2026-05-24T00:00:00%2B09:00&
  air_time=lt.2026-05-25T00:00:00%2B09:00&
  select=*,drama:dramas(id,title,poster_url,channel:channels(code,name))&
  order=air_time.asc

# 즐겨찾기 추가
POST /rest/v1/favorites
{ "user_id": "...", "drama_id": "..." }
```

### 5-2. Edge Function이 필요한 케이스 (직접 설계)

PostgREST로 어려운 것: 트랜잭션, 외부 API 호출, 무거운 집계, 푸시 발송.

```
POST /functions/v1/toggle-favorite
Authorization: Bearer <user_jwt>
{ "drama_id": "uuid" }

Response 200:
{
  "favorited": true,
  "push_jobs_scheduled": 12  // 즐겨찾기와 동시에 향후 12회차 알림 예약
}
```

```
GET /functions/v1/home-feed
Authorization: Bearer <user_jwt>

Response 200:
{
  "on_air_now": [...],          // 지금 방영중
  "starts_soon": [...],        // 1시간 이내 시작
  "favorites_today": [...],    // 즐겨찾기 중 오늘 방영
  "hot_posts": [...]
}
```

```
POST /functions/v1/posts/{post_id}/report
{
  "reason": "SPOILER",
  "detail": "회차 정보에 결말 노출"
}
```

### 5-3. API 응답 컨벤션

```
// 성공
{ "data": { ... } }

// 페이지네이션 (커서 기반)
{
  "data": [...],
  "next_cursor": "eyJjcmVhdGVkX2F0IjoimjAyNi0wNS0yNFQxMjowMDowMFoifQ=="
}

// 에러
{
  "error": {
    "code": "FAVORITE_LIMIT_EXCEEDED",
    "message": "즐거찾기는 최대 100개까지 가능합니다."
  }
}
```

페이지네이션은 cursor 기반으로 처음부터. 인스타·트위터식 무한스크롤에서 offset은 LIMIT 20 OFFSET 10000 같은 쿼리에서 OOM을 만듭니다.

## 6. 추천 기술 스택

### 6-1. 백엔드 비교 (1인 개발 + 한국 시장 기준)

| 항목           | Firebase           | Supabase ★             | Spring Boot + MySQL | NestJS + Postgres |
|--------------|--------------------|------------------------|---------------------|-------------------|
| 초기 개발 속도     | ★★★★★              | ★★★★★                  | ★★                  | ★★★               |
| 학습 비용 (1인)   | 낮음                 | 낮음                     | 매우 높음               | 중간                |
| 관계형 쿼리       | ✗ (NoSQL)          | ✓                      | ✓                   | ✓                 |
| 한국 데이터 주권    | 미국 리전              | Seoul ap-northeast-2 ✓ | 자유                  | 자유                |
| 푸시           | FCM 통합             | APNs 직접                | 직접                  | 직접                |
| 인증 (Apple)   | ✓                  | ✓                      | 직접 구현               | 직접 구현             |
| 실시간          | Firestore listener | Realtime (CDC)         | 직접 WebSocket        | 직접 WebSocket      |
| 1만 MAU 월 비용  | \$25~              | \$25 (Pro)             | \$30~50             | \$30~50           |
| 10만 MAU 월 비용 | \$200~500          | \$100~300              | \$100~200           | \$100~200         |
| 벤더 락인        | 매우 높음              | 낮음 (Postgres 표준)       | 없음                  | 없음                |

최종 추천: Supabase로 시작 → 트래픽이 Pro 한도(8GB DB / 250GB egress)를 넘으면 일부 워크로드만 분리.

#### 이유:

1. **Postgres 표준**: 나중에 AWS RDS로 옮기는 게 SQL dump 한 번이면 됩니다. Firestore에서 RDB로 옮기는 건 거의 재작성입니다.
2. **관계형 쿼리**: 드라마-회차-즐거찾기-알림은 본질적으로 관계형입니다. Firestore에서 "내 즐겨찾기 드라마의 오늘 방영 회차"를 뽑는 건 N+1 지옥입니다.
3. **RLS만 잘 짜면 백엔드 코드가 거의 0**: 1인 개발자에게 결정적입니다.
4. **Edge Function(Deno/TypeScript)**으로 복잡한 로직만 처리.

다음 단계 트리거: MAU 5만 이상 + Edge Function이 cold start로 느려지면 → 핫 패스만 **NestJS + Fly.io**로 빼냅니다.  
(Spring Boot는 1인 운영 부담이 큼)

## 6-2. iOS 의존성 (최소)

| 라이브러리                       | 용도             | 대안                    |
|-----------------------------|----------------|-----------------------|
| <code>supabase-swift</code> | API/Auth 클라이언트 | URLSession 직접         |
| <code>Kingfisher</code>     | 이미지 캐싱         | AsyncImage (성능↓)      |
| <code>SwiftData</code>      | 로컬 즐겨찾기/캐시     | CoreData              |
| Apple Sign In               | 로그인            | (Apple Native)        |
| 금지: Alamofire, RxSwift      |                | Swift Concurrency로 충분 |

처음부터 의존성은 5개 이하로 유지하세요. 라이브러리 하나가 Swift 7 호환 안 되면 발목 잡힙니다.

## 7. 서버 아키텍처

### 7-1. 단계별 진화

[Phase 1: 0~5천 MAU]

iOS App → Supabase (DB+Auth+EdgeFunc+Storage)

↑  
└─ Crawler (Fly.io 1 VM, \$2/월)

[Phase 2: 5천~5만 MAU]

iOS App → CloudFront/CloudFlare CDN → Supabase

↑  
└─ Crawler Worker (Cloud Run)  
↑  
└─ Redis (Upstash) 캐시 레이어

[Phase 3: 5만+ MAU]

iOS App → CDN → API Gateway

└─ Supabase (인증/RLS)  
└─ NestJS API (홈피드/인기글 등 핫 패스)  
└─ Push Worker (별도 컨테이너, APNs HTTP/2 풀)  
Redis Cluster + 이벤트 큐(SQS)

### 7-2. 크롤러 분리가 필수인 이유

크롤러는 DB와 같은 머신에 두면 안 됩니다.

- 차단당해서 IP 바뀌어야 할 때 DB까지 영향
- 크롤러 죽으면 앱이 죽는 일 방지
- 별도 cron 스케줄링 (매시간 / 매일 새벽 3시 전체 동기화)

Fly.io의 `auto_stop_machines = true` 옵션을 쓰면 크롤링 안 할 때는 머신이 꺼져서 월 \$1~3 수준입니다.

### 7-3. 캐시 전략

| 데이터        | 캐시 위치            | TTL        | 이유               |
|------------|------------------|------------|------------------|
| 오늘 편성표     | CDN + 클라이언트      | 10분        | 거의 안 바뀜, 트래픽 90% |
| 드라마 메타     | 클라이언트(SwiftData) | 1일         | 포스터/제목 잘 안 바뀜    |
| 사용자 즐겨찾기   | 클라이언트(SwiftData) | 무한 (서버 동기) | 오프라인에서도 보여야 함    |
| 인기글 Top 10 | Redis            | 5분         | 계산 비용 큼          |
| 게시글 상세     | CDN (비인증)        | 1분         | 새 댓글 반영 빠르게      |

오늘 편성표 1건 = 모든 사용자에게 동일입니다. 사용자 1만명이 각자 DB 쿼리하면 안 됩니다. CDN edge에서 잘라야 합니다.

## 8. 푸시 알림 구조

### 8-1. 가장 큰 문제: "1만 명에게 동시에 푸시 발송"

방영 시작 10분 전, 즐겨찾기 5천 명에게 동시 발송하면:

- APNs는 HTTP/2로 stream 다중화. 한 connection으로 초당 ~1000건.
- Edge Function의 timeout(30초)을 넘기기 쉬움.
- 실패한 토큰 재시도 필요.

### 8-2. 권장 구조

```
[10분마다 cron]
↓
push_jobs 테이블에서 scheduled_at <= now() + 30s AND status = 'PENDING' 조회
↓
배치로 1000건씩 묶어 Push Worker로 전송 (이벤트 큐)
↓
Push Worker: APNs HTTP/2 connection pool, 동시 100 streams
↓
성공: status = 'SENT'
실패 (410 Unregistered): devices.apns_token 무효화
실패 (다른): 재시도 큐 (max 3회)
```

### 8-3. 푸시 잡 미리 생성 (Pre-scheduling)

크롤러가 새 회차를 발견하면, 즉시 모든 즐겨찾기 사용자의 `push_jobs` 를 미리 만들어둡니다:

```
-- 새 회차 등록 시 트리거
insert into push_jobs (user_id, episode_id, scheduled_at, payload)
select
  f.user_id,
  e.id,
  e.air_time - interval '10 minutes',
  jsonb_build_object(
    'aps', jsonb_build_object(
      'alert', jsonb_build_object(
        'title', d.title || ' ' || e.number || '회',
        'body', '10분 후 방영 시작!'
      ),
      'sound', 'default',
      'category', 'EPISODE_AIR'
    ),
    'drama_id', d.id,
    'episode_id', e.id
  )
from episodes e
join dramas d on d.id = e.drama_id
join favorites f on f.drama_id = d.id and f.notify_air = true
where e.id = NEW.id;
```

이렇게 하면 발송 시점에는 `select ... where scheduled_at <= now()` 한 줄 쿼리로 끝납니다.

## 8-4. 알림 카테고리 (iOS)

```
UNNotificationCategory(
  identifier: "EPISODE_AIR",
  actions: [
    UNNotificationAction(identifier: "OPEN_DRAMA", title: "보러가기"),
    UNNotificationAction(identifier: "SNOOZE", title: "10분 후 다시"),
  ],
  intentIdentifiers: []
)
```

딥링크 페이로드( `drama_id` )로 상세 화면 직행.

## 8-5. APNs 인증

.p8 키 방식 강력 추천 (인증서 방식 X). 키는 1년 안 만료, 모든 앱 공유 가능. Supabase Edge Function에 환경변수로 주입.

## 9. 커뮤니티 설계

커뮤니티는 반드시 터집니다. 다음을 처음부터 막아주세요.

### 9-1. 반드시 터지는 문제 5가지

| 문제           | 발생 시점        | 대응                           |
|--------------|--------------|------------------------------|
| 스포일러 신고 폭주   | 첫 방영 직후 30분  | 회차별 스포일러 토글 필수, 기본값 ON       |
| 여혐/남혐 댓글 전쟁  | 인기 드라마 출시 직후 | 신고 3회 자동 가림, 24시간 내 처리       |
| 타팬덤 침공/평점 테러 | 인기글 댓글창      | 가입 7일 미만 신규 계정 댓글 제한 옵션      |
| 광고/도배 봇      | 사용자 1만 돌파 직후 | 1분 내 3회 도배 시 자동 차단, hCaptcha |
| 자살/자해 게시물    | 비주기적         | 키워드 자동 hide + 안내 메시지         |

### 9-2. 스포일러 블라인드

```
struct PostRow: View {
    let post: Post
    @State private var revealed = false

    var body: some View {
        VStack(alignment: .leading) {
            Text(post.title)
            if post.isSpoiler && !revealed {
                Text("🚫 스포일러 - 탭하여 보기")
                    .padding().frame(maxWidth: .infinity)
                    .background(.gray.opacity(0.3))
                    .onTapGesture { revealed = true }
            } else {
                Text(post.body)
            }
        }
    }
}
```

회차 정보( `episode_no` )와 함께 저장해서, "내가 본 회차 이전 스포만 보기" 옵션도 v2에서 추가 가능.

### 9-3. 인기글 점수 (Hacker News 식)

```
hot_score = (likes - 1) / pow((age_in_hours + 2), 1.5)
```

- pg\_cron으로 5분마다 최근 24시간 게시글 재계산
- `idx_posts_hot` 로 인덱스 정렬
- 인기글 노출은 Top 20만, 그 이상은 의미 없음

## 9-4. 신고 자동화

```
-- 같은 게시글에 신고 3회 누적 시 자동 hide
create or replace function auto_hide_on_report() returns trigger as $$
begin
    if (select count(*) from reports
        where target_type = NEW.target_type
        and target_id = NEW.target_id
        and status = 'PENDING') >= 3 then
        if NEW.target_type = 'POST' then
            update posts set is_hidden = true where id = NEW.target_id;
        end if;
    end if;
    return NEW;
end;
$$ language plpgsql;
```

자동 가림 후 운영자 검토 → 복원 또는 영구 삭제. Apple 가이드 1.2(UGC)는 **24시간 내 처리**를 요구합니다.

# 10. 드라마 데이터 수집 전략

이 부분이 법적/기술적으로 가장 까다롭습니다.

## 10-1. 데이터 소스 매트릭스

| 소스                  | 공식 API  | 무료 | 안정성   | 법적 리스크     | 추천도     |
|---------------------|---------|----|-------|------------|---------|
| TMDB                | ✓       | ✓  | ★★★★★ | 낮음 (출처 표기) | ★★★★★   |
| KOFIC (영화)          | ✓       | ✓  | ★★★★  | 낮음         | ★★★     |
| KBS/MBC/SBS         | ✗       | -  | ★★    | 중간         | 크롤링     |
| tvN/JTBC (CJ ENM)   | ✗       | -  | ★     | 중간         | 크롤링     |
| EPG XML (skylife 등) | 일부      | 유료 | ★★★   | 낮음 (계약 시)  | 자금 있을 때 |
| Netflix/Disney+     | ✗ (비공개) | -  | -     | 높음         | 비추      |
| Wavve/Tving         | ✗       | -  | -     | 높음         | 비추      |

## 10-2. 현실적 수집 전략 (단계별)

Phase 1 (MVP): TMDB + 지상파 크롤링

- TMDB로 드라마 메타데이터(포스터, 시놉시스, 출연진) 확보
- KBS/MBC/SBS 편성표 페이지 크롤링 (HTML 파싱)
- 일 1회 새벽 3시 + 매시간 보정 크롤

Phase 2: + 종편/케이블

- tvN, JTBC, ENA, 채널A 등 추가
- 사용자 신고 기반 보정 ("편성표 오류" 버튼)

Phase 3: OTT 확장

- 사용자가 직접 "이 드라마는 넷플릭스에서" 태깅 (UGC 보장)
- Netflix는 공식 API 없음 → 크롤링은 ToS 위반 위험

### 10-3. 크롤러 코드 패턴 (TypeScript / Cloud Run)

```
// crawler/src/kbs.ts
import { chromium } from 'playwright'

export async function crawlKBS(targetDate: Date) {
  const browser = await chromium.launch({ headless: true })
  const page = await browser.newPage({
    userAgent: 'Mozilla/5.0 (DramaApp Crawler; contact@dramaapp.com)',
    locale: 'ko-KR',
  })

  await page.goto(`https://program.kbs.co.kr/2tv/schedule/${formatDate(targetDate)}`)
  await page.waitForSelector('.schedule-item', { timeout: 10000 })

  const items = await page.$$eval('.schedule-item', (els) =>
    els.map(el => ({
      time: el.querySelector('.time')?.textContent?.trim(),
      title: el.querySelector('.title')?.textContent?.trim(),
      episode: el.querySelector('.episode')?.textContent?.trim(),
    }))
  )

  await browser.close()
  return items.map(normalizeKBS)
}
```

### 10-4. 크롤링 차단 대응 전략

| 차단 유형           | 대응                                                 |
|-----------------|----------------------------------------------------|
| User-Agent 차단   | 정직한 UA + 연락처 명시 ("DramaApp Crawler; contact@...")  |
| Rate limit      | 요청 간 2~5초 sleep, 분산 시간대                            |
| IP 차단           | 주거용 프록시 (\$50/월) — 단, ToS 회색지대                     |
| Cloudflare 봇 차단 | Playwright + <code>playwright-extra-stealth</code> |
| JS 렌더링 필수       | 위 동일, headless Chrome                              |
| HTML 구조 변경      | 모니터링 alert + 빠른 패치가 더 중요                           |

가장 자주 깨지는 건 차단이 아니라 HTML 구조 변경입니다. 셀렉터 깨지면 Slack/Discord webhook으로 즉시 알람. 일일 크롤링 결과를 전일 대비 ±20% 넘으면 의심.

## 10-5. 법적 리스크 정리

| 항목            | 한국법                  | 실무적 위험도               |
|---------------|----------------------|-----------------------|
| 편성표 크롤링       | 저작권법 보호 대상 X (사실 정보) | 낮음                    |
| 시놉시스 크롤링      | 보호 대상 O (창작물)        | 중간 — TMDB만 사용 권장      |
| 포스터 크롤링       | 보호 대상 O              | 높음 — TMDB만            |
| robots.txt 무시 | 직접적 위법 X, ToS 위반 가능  | 중간                    |
| 부정경쟁방지법(2021) | "성과 도용" 조항           | 잠재적 고위험 — 대형 사업자 분쟁 시 |

### 실전 가이드라인

1. 편성 시간만 크롤링, 시놉시스/포스터는 TMDB API만 사용
2. robots.txt 준수, 1초당 1요청 이하
3. 출처 표기 ("편성 정보 제공: KBS/MBC/...")
4. 방송사가 공식 요청 시 즉시 중단할 수 있는 컨택 채널 제공
5. 부정경쟁방지법 리스크: tvN 편성표만 그대로 가져와서 보여주면 위험. 자체 부가가치(즐거찾기/알림/커뮤니티) 명확히 할 것

# 11. 예상 트래픽 고려사항

## 11-1. 시나리오별 추정

| 지표                | 1천 MAU | 1만 MAU  | 10만 MAU   |
|-------------------|--------|---------|-----------|
| DAU (DAU/MAU 25%) | 250    | 2,500   | 25,000    |
| 일 세션 (2회/DAU)     | 500    | 5,000   | 50,000    |
| API 요청/세션         | 20     | 20      | 20        |
| 일 API 요청          | 10,000 | 100,000 | 1,000,000 |
| 피크 RPS (3배 spike) | 0.5    | 5       | 50        |
| DB 크기             | 100MB  | 500MB   | 5GB       |
| 푸시 발송/일           | 1,000  | 10,000  | 100,000   |
| 이미지 egress/월      | 5GB    | 50GB    | 500GB     |

## 11-2. 병목 지점 예측

| MAU  | 첫 병목             | 해결                      |
|------|------------------|-------------------------|
| 5천   | 이미지 egress 비용    | CloudFlare CDN 무료 플랜 앞단 |
| 1만   | 편성표 API 캐시 미스    | Edge cache 10분 TTL      |
| 5만   | 푸시 발송 동시성        | 큐 + Worker 분리           |
| 10만  | Supabase 단일 인스턴스 | 읽기 전용 replica or 캐시 적극화 |
| 30만+ | DB 쓰기 (게시글/좋아요)  | Hot 게시글만 Redis로 카운터     |

## 12. 비용 절감 전략

### 12-1. MAU별 예상 월 비용

|            |                                                                                                                                 |
|------------|---------------------------------------------------------------------------------------------------------------------------------|
| [1천 MAU]   | Supabase Free \$0 + Fly.io \$3 + CloudFlare \$0 = \$3/월                                                                         |
| [1만 MAU]   | Supabase Pro \$25 + Fly.io \$5 + CloudFlare \$0 = \$30/월                                                                        |
| [10만 MAU]  | Supabase Pro+addon \$80 + Cloud Run \$20 +<br>Upstash Redis \$10 + CloudFlare Pro \$20 +<br>APNs \$0 (무료) + 모니터링 \$10 = \$140/월 |
| [100만 MAU] | \$1,000~2,000/월 (이 시점엔 광고/구독 수익으로 흑자)                                                                                           |

### 12-2. 절감 포인트 (효과 큰 순)

1. 이미지: 절대로 Supabase Storage egress 직접 노출 X — 앞에 CloudFlare CDN 무조건. Egress 비용이 전체의 60%를 차지함.
2. 포스터는 TMDB CDN 직링크 사용 (TMDB가 무료 CDN 제공) — Storage 비용 0
3. 편성표 캐시 TTL 길게 (10분) — 같은 응답을 N명이 받아감
4. 이미지 변환은 클라이언트에서 — 서버 리사이즈는 비쌈, SwiftUI `.resizable()` + Kingfisher
5. 푸시 토큰 정리 — 무효 토큰 누적되면 APNs HTTP 호출 낭비. 410 응답 즉시 삭제
6. 콜드 스타트 회피 — Edge Function 자주 호출되는 건 keep-alive ping

## 13. 추후 수익화 아이디어

| 모델                      | 시점   | 예상 수익 (10만 MAU)              | 리스크                  |
|-------------------------|------|------------------------------|----------------------|
| 배너 광고 (AdMob)           | v1.1 | 월 \$200~500                  | UX 저하, 4.3 리젝 가능성 낮음 |
| 네이티브 광고 (편성표 사이)        | v2   | 월 \$500~1500                 | 잘 만들면 OK             |
| 드라마 굿즈 제휴 (쿠팡파트너스)      | v2   | 월 \$100~300                  | 트래픽 의존               |
| OTT 가입 제휴 (웨이브/티빙)      | v2   | 건당 \$5~10                    | 협상 필요                |
| 프리미엄 구독 (광고 제거 + 한정 기능) | v3   | \$2.99/월, 전환율 2% = 월 \$5,980 | 차별화 어려움              |
| 드라마 굿즈 자체 판매            | v4+  | -                            | 운영 부담 큼              |
| 응원 후원/도네이션              | v3   | 미미                           | 한국 정서상 약함            |

**현실 조언:** 10만 MAU 전까지는 광고 수익이 운영비를 못 넘깁니다. 광고는 **생존 도구**가 아니라 **재투자 도구**로 생각하세요. 진짜 수익은 OTT 제휴/굿즈입니다.

## 14. 앱스토어 출시 시 주의사항

### 14-1. 자주 리젝되는 항목 (이 앱 기준)

| 가이드                       | 내용                                                | 대응                        |
|---------------------------|---------------------------------------------------|---------------------------|
| 1.2 UGC                   | UGC 앱은 (1) 신고 (2) 차단 (3) 24시간 내 처리 (4) 이용약관 동의 필수 | 출시 전 4종 모두 구현             |
| 2.5.2 Code Download       | 런타임 코드 다운로드 금지                                    | Edge Function 응답이 데이터면 OK |
| 3.1.1 IAP                 | 디지털 콘텐츠는 IAP만                                     | 광고 제거는 IAP 필수             |
| 4.3 Spam                  | 다른 편성표 앱과 차별화 부족 시                                | 커뮤니티/알림으로 차별화 명시          |
| 4.7 (한국)                  | 청소년 보호                                            | 17+ 등급 권장 (UGC)           |
| 5.1.1 Data Collection     | 추적 시 ATT 프롬프트 필수                                  | AdMob 사용 시 필수             |
| 5.1.1(v) Account Deletion | 2022년 6월부터 회원 탈퇴 앱 내 구현 필수                        | 마이 화면에 명시적 버튼             |
| 5.1.4 Kids                | 만 13세 미만 대상 X (한국 14세 미만)                         | 4+ 등급 시 트래킹 금지            |

### 14-2. 리뷰 통과 팁

1. **데모 계정 제공**: 로그인 필요 화면이 있으면 리뷰 노트에 Apple ID 데모 계정 명시
2. **첫 출시는 기능 최소화**: 커뮤니티 빼고 편성표/즐거찾기/알림만으로 v1.0 출시 → 4.3(spam) 리젝 가능성 줄임 + UGC 가이드 우회
3. **편성 정보 출처 명시**: "방송사 공식 사이트 기반" 푸터 표기
4. **테스트 디바이스 영상 캡처**: 푸시 알림 동작 영상 미리 준비

### 14-3. 한국 특수

- 정보통신망법: 14세 미만 회원가입 시 법정대리인 동의 필요 → **차라리 14+ 제한**
- 약관/개인정보처리방침: 한국어 + 영어 둘 다 필수
- 게시판 운영자 표시 의무: 마이 → 회사 정보에 운영자/연락처 명시

## 15. 실제 서비스 운영 시 가장 어려운 점

15년 경험상 이 5개에서 99%가 죽습니다.

| 문제               | 발생 빈도          | 1인 대응 가능? | 대비책                                              |
|------------------|----------------|-----------|--------------------------------------------------|
| 크롤러가 새벽에 깨짐      | 월 2~5회         | 어려움       | 모니터링 + Slack 알림 + fallback (전일 데이터 유지)           |
| 푸시가 안 옴 / 늦게 옴   | 주 1회 사용자 CS    | 가능        | 발송 로그 보관 ( <code>push_jobs.sent_at</code> ), CS툴 |
| 편성 시간 오류 → 폭주 CS | 월 1~2회         | 가능        | "오류 신고" 버튼 + 자동 보정 워크플로우                         |
| 커뮤니티 욕설/스포 신고 폭주 | 인기드라마 첫방 후 30분 | 불가능       | 자동 hide + 가입 7일 제한 + 키워드 필터                      |
| 부정경쟁/저작권 내용증명    | 1년 1~2회        | 어려움       | 변호사 콘택트 미리, 즉시 비활성화 가능한 admin flag               |

핵심 깨달음: 코드 99%가 잘 굴러가도, 운영 1%가 무너지면 앱이 죽습니다. 운영 자동화에 개발의 30%를 써야 합니다.

## 16. 혼자 개발할 경우 현실적인 개발 순서

---

경험칙: 1인 개발자는 "기능 완성도 60% × 출시 빈도 높음"이 "100% × 6개월 지연"을 이깁니다.

## 16-1. 주차별 작업

### [Week 1]

- Supabase 프로젝트 생성 + DB 스키마 (DDL 위 그대로 적용)
- TMDb API 키 발급, 채널 목록 시드
- Xcode 프로젝트 SwiftUI로 재구성 (스토리보드 제거)
- 폴더 구조 + DesignSystem 토큰

### [Week 2]

- 크롤러 1개 (KBS만) 작성, Fly.io 배포
- cron으로 매일 새벽 3시 실행 확인
- dramas/episodes 테이블에 실데이터 100건 채우기

### [Week 3]

- iOS: 편성표 화면 (날짜/채널 선택)
- SupabaseDramaRepository 구현
- 로딩/에러 상태 처리

### [Week 4]

- 즐겨찾기 (로컬 SwiftData) - 로그인 없이도 작동
- 드라마 상세 화면

### [Week 5]

- Apple Sign In
- 게스트 → 가입자 데이터 마이그레이션
- 즐겨찾기 서버 동기

### [Week 6]

- Push 등록 (디바이스 토큰 저장)
- push\_jobs 트리거 작성
- Push Worker (Edge Function) 구현
- 본인 디바이스로 발송 테스트

### [Week 7]

- MBC, SBS 크롤러 추가
- 편성표 캐시 (CDN/Edge)
- 검색 기능

### [Week 8]

- 마이 화면 + 회원탈퇴
- 약관/개인정보처리방침 (Notion 페이지로 호스팅)
- TestFlight 베타 (지인 20명)

### [Week 9-10]

- 베타 피드백 반영
- 충돌/오류 수집 (TelemetryDeck or Sentry)
- 앱스토어 메타데이터 (스크린샷 6장, 설명, 키워드)

### [Week 11]

- App Store 제출 (커뮤니티 없는 v1.0)
- 리뷰 통과까지 평균 1~3일

### [Week 12]

- 출시 후 모니터링
- 사용자 100명 도달 시 → 다음 단계 (커뮤니티) 결정

## 17. 3개월 개발 로드맵

---

### Month 1: "혼자 보는 드라마 캘린더" 출시 가능 수준

- DB + 크롤러(지상파 3사) + iOS 편성표/즐거찾기/푸시
- **출구 조건:** 본인이 매일 쓸 수 있고, 푸시 알림이 정확함
- **결과물:** TestFlight 베타 가능한 v0.9

### Month 2: 출시 & 안정화

- Apple Sign In, 회원 탈퇴, 약관
- 종편/케이블 크롤러 (tvN, JTBC, ENA)
- 검색, 장르 필터
- **출구 조건:** App Store 출시 + DAU 100명 + 크래시율 0.5% 이하
- **결과물:** App Store v1.0 출시

### Month 3: 커뮤니티 베타 + 데이터 기반 의사결정

- 커뮤니티 (게시판/댓글/좋아요/신고/차단/블라인드)
- 운영자 admin 패널 (Retool 또는 Supabase Studio + RLS bypass key)
- 분석 (PostHog 또는 Mixpanel 무료)
- **출구 조건:** 게시글 일 30개, 신고 자동처리 동작
- **결과물:** v1.5 커뮤니티 활성화

#### 3개월 후 의사결정 분기점

- DAU 300명 미만 → 마케팅 + 차별화 기능 (커뮤니티 강화)
- DAU 1000명 이상 → 광고 도입 + 인프라 확장

## 18. 확장 가능한 구조 설계 방법

지금 안 해도 되지만, 나중에 갈아엎지 않으려면 처음부터 지킬 5가지:

### 18-1. 인터페이스로 끊어둘 곳

```
// 처음부터 프로토콜로 정의
protocol DramaRepository {
    func schedule(date: Date, channelIds: [UUID]?) async throws -> [Episode]
    func toggleFavorite(dramaId: UUID) async throws -> Bool
}

// MVP 구현
final class SupabaseDramaRepository: DramaRepository { ... }

// 나중에 NestJS로 교체 시
final class NestJSDramaRepository: DramaRepository { ... }
// ViewModel은 한 줄도 안 바뀜
```

### 18-2. 도메인 이벤트 (나중에 큐로)

```
// 지금은 함수 직접 호출
async function onEpisodeCreated(episode) {
    await scheduleNotifications(episode)
    await invalidateCache(episode.dramaId)
}

// 사용자 많아지면 이벤트 버스(SQS/Pub-Sub)로 분리
await eventBus.publish('episode.created', { episodeId: episode.id })
```

이벤트 이름을 처음부터 명사+과거형( `episode.created` )으로 잡아두면 그대로 큐로 옮길 수 있습니다.

### 18-3. Feature Flag 처음부터

```
enum FeatureFlag {
    static var communityEnabled: Bool {
        RemoteConfig.shared.bool("community_enabled", default: false)
    }
}
```

서버 사이드( `app_config` 테이블)에 토글을 두고, 앱 업데이트 없이 기능을 끌 수 있어야 합니다. 커뮤니티가 터졌을 때 30초 안에 잠가야 합니다.

## 18-4. 멀티 채널 → 멀티 국가 확장

`channels.type` 을 `'BROADCAST' | 'OTT'` 로 시작했지만, `country_code` 컬럼을 처음부터 추가하세요. 일본/대만 드라마 시장은 다음 단계 시장입니다.

## 18-5. 모듈화 타이밍

[Phase 1] 단일 앱 타겟 (현재)  
[Phase 2] WidgetKit Extension (오늘 편성 위젯) → Shared/ 모듈 분리  
[Phase 3] watchOS 앱 → Domain 패키지 SPM 분리  
[Phase 4] 빌드 시간 3분 초과 → Feature별 SPM 패키지

Domain 폴더의 모델/리포지토리 프로토콜은 Foundation 외 import를 금지하세요. 이게 미래의 모듈화를 가능하게 합니다.

## 19. 최종 요약 — "오늘 당장 할 일"

---

1. `DramaApp.xcodeproj` 를 SwiftUI로 재구성: 스토리보드 제거, `@main DramaAppApp` 생성, 탭 5개 골격
2. Supabase 프로젝트 생성 (Seoul 리전), 위 DDL 그대로 적용
3. TMDB API 키 발급 (무료)
4. `channels` 테이블 시드: KBS2, MBC, SBS, tvN, JTBC 5개
5. `DramaRepository` 프로토콜 먼저 정의, Mock 구현으로 UI부터

여기까지가 첫 주 작업입니다.

---

### 가장 강조하고 싶은 3가지

1. 혼자 만들 수 있는 가장 큰 차별화는 "정확한 푸시 알림"입니다. 편성표만 있는 앱은 많지만 알림이 정확하지 않습니다. 여기에 1주를 더 써도 됩니다.
  2. 커뮤니티를 너무 빨리 열지 마세요. 사용자 1만 명 이하에서 게시판은 죽은 공간이고, 죽은 공간은 앱의 평판을 깎습니다.
  3. 크롤러 모니터링이 앱 코드보다 중요합니다. 새벽에 크롤러가 깨지면 사용자 1만 명이 동시에 잘못된 알림을 받습니다. Slack webhook 알림은 1일차에 박으세요.
-